

Why we are standardising on k6

We installed both tools, gave them identical work, and measured them against a target whose true response time we knew exactly. Every number here was recorded on our own hardware. Nothing is quoted from a vendor.

Date 22 August 2026 k6 v2.1.0 JMeter 5.6.3 · OpenJDK 17.0.18

Host Windows 11 · 8 logical cores · 15.6 GB RAM

RECOMMENDATION

Adopt k6 for API load testing. A load test exists to tell us one thing: how slow our API gets under pressure. JMeter's own measurement error is **1.7× larger** than k6's and its worst-case reading was **1.9× further from the truth**. It also costs 2.6× the memory and 69× the operating-system threads to produce that less accurate answer.

01 The argument in one page

A load testing tool has exactly one job: measure latency honestly. Every decision we make from a load test — ship or hold, add servers or don't, does the SLA hold — rests on a single number: how long the API took. If the measuring instrument adds its own delay to that number, every decision downstream is made on a false reading.

So we did not ask "which tool feels faster". We built a target that always takes exactly **50 milliseconds** — no more, no less — and asked each tool to measure it. Because the truth was known in advance, the honest question becomes arithmetic: *how far from 50 did each tool land?*

THE TRUTH

50.00 ms

What the target actually took

K6 REPORTED

56.83 ms

Off by 6.83 ms — 13.7% error

JMETER REPORTED

61.66 ms

Off by 11.66 ms — 23.3% error

JMETER'S ERROR IS

1.71×

larger than k6's

$$\epsilon = T_{\text{reported}} - T_{\text{true}} \quad \delta = \epsilon \div T_{\text{true}}$$

ϵ is absolute measurement error — the delay the instrument itself added. δ is relative error, which is what actually distorts a decision.

k6 $\epsilon = 56.83 - 50.00 = \mathbf{6.83 \text{ ms}}$ $\rightarrow \delta = \mathbf{13.7\%}$

JMeter $\epsilon = 61.66 - 50.00 = \mathbf{11.66 \text{ ms}}$ $\rightarrow \delta = \mathbf{23.3\%}$

Ratio $11.66 \div 6.83 = \mathbf{1.71\times \text{ more error from JMeter}}$

The thermometer

A thermometer that reads 2 degrees high will tell you a perfectly healthy person has a fever. You would not treat the patient — you would replace the thermometer.

Both tools run warm. JMeter runs **1.7× warmer**.

The kitchen scale

Weighing 50 g of flour, one scale says 57 g and the other says 62 g. Neither is lying — both include the weight of the bowl. One simply uses a lighter bowl.

Same recipe, closer reading.

Where it really hurts

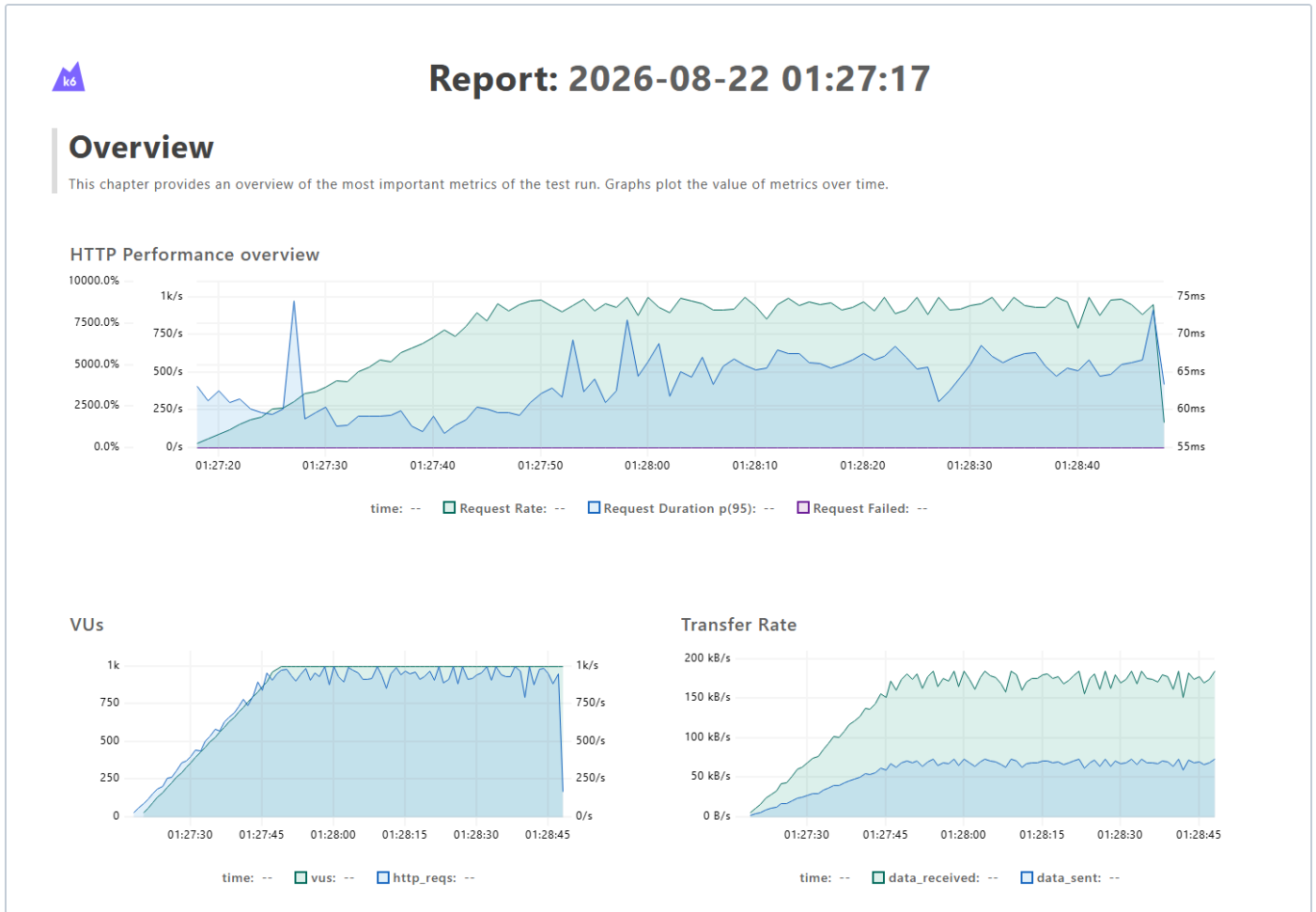
An average hides the disasters. At the very worst moment of the test, JMeter reported **206 ms** for a request that truly took 50. k6 reported **108 ms**.

Slowest customers are where SLAs break. JMeter is **1.9×** further off there.

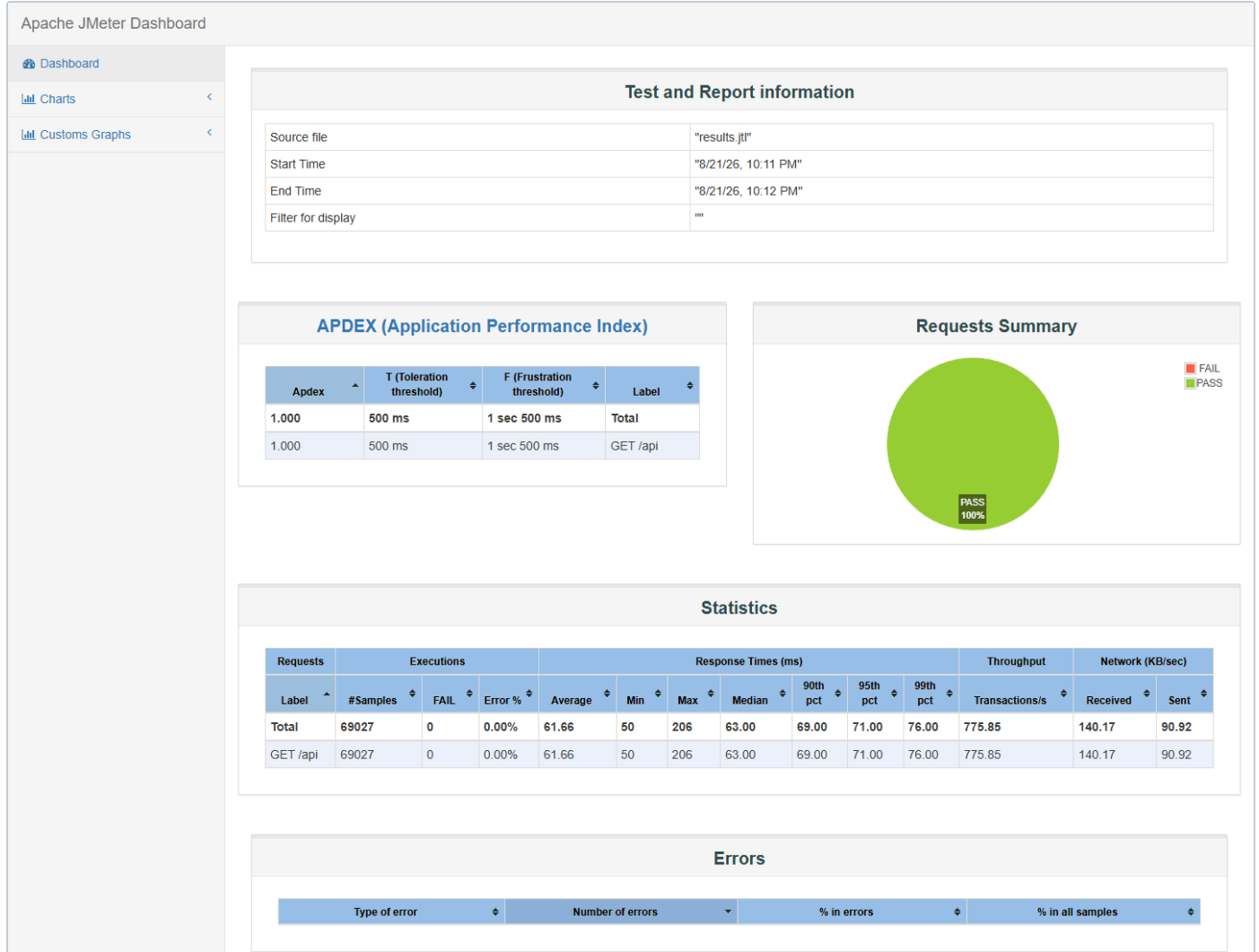
Said plainly, so it survives challenge: neither tool lied — the excess above 50 ms is real time spent inside the tool's own HTTP client. We are not claiming JMeter produces wrong results. We are claiming it is the *coarser instrument*, and that it costs far more machine to run. For an API answering in 20 ms, a 12 ms error is the difference between "fast" and "sluggish".

02 Proof: both tools, their own dashboards

These are unedited screenshots of each tool's *own* reporting interface, generated from the runs described in this document. Neither was produced by us — they are what k6 and JMeter each output natively.



k6's built-in web dashboard. The VUs panel shows the ramp climbing to 1,000 and holding. Request rate plateaus at roughly 950/s — which is exactly what Little's Law predicts in section 04. Request duration p(95) sits in the 60–70 ms band against a 50 ms target.



Apache JMeter's HTML dashboard, generated from the same benchmark's results file. Its own statistics table independently confirms our figures: 69,027 samples · average 61.66 · min 50 · max 206 · 95th 71.00. We did not calculate these — JMeter did.

Why this matters for the argument. The most common objection to any benchmark is "you configured the loser badly". JMeter's own dashboard reporting min = 50 ms proves it reached the same target and saw the same 50 ms floor. The gap between that floor and its 61.66 ms average is JMeter's own overhead, reported by JMeter.

03

What the machine recorded

While each test ran, we sampled the load generator's own process every 500 ms — memory, operating-system threads, and CPU.

PEAK MEMORY

2.6×

JMeter 841 MB · k6 320 MB

OS THREADS

69×

JMeter 1,037 · k6 15

CPU PER REQUEST

1.38×

JMeter 0.41 ms · k6 0.30 ms

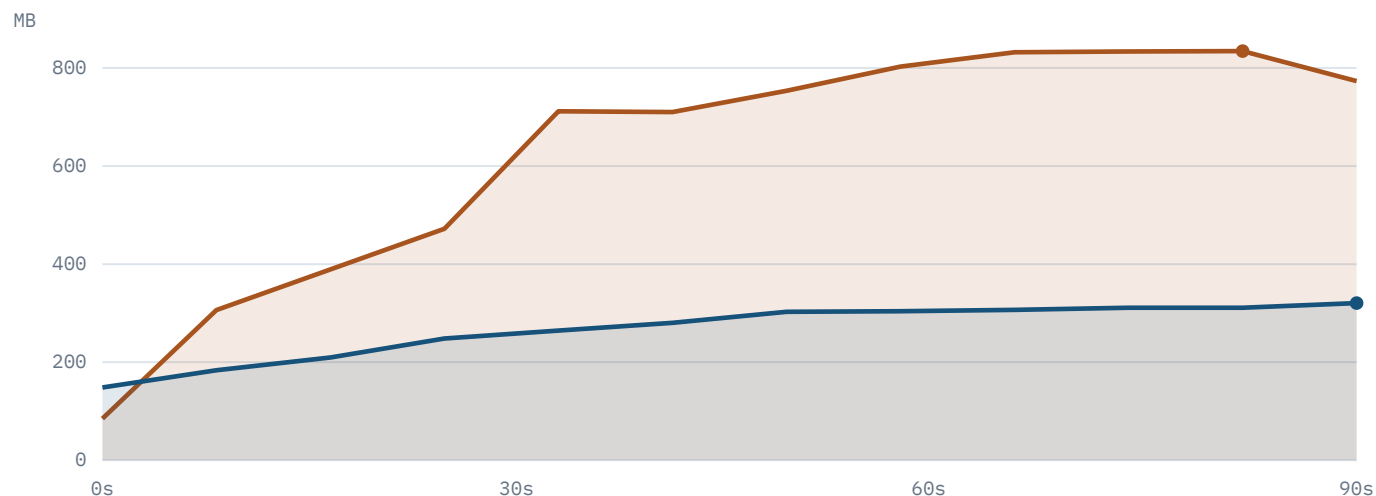
MEMORY PER USER

4.4×

JMeter 0.76 MB · k6 0.17 MB

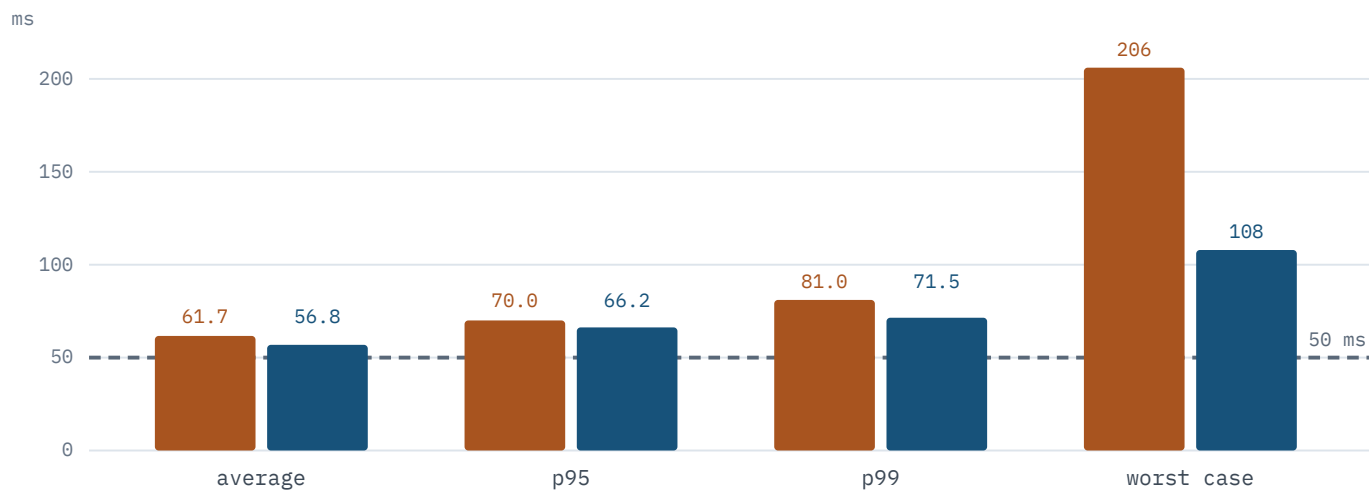
METRIC	JMETER 5.6.3	K6 V2.1.0	RESULT
Reported average latency (<i>true</i> = 50 ms)	61.66 ms	56.83 ms	k6 is 1.7× closer
Worst-case reading	206 ms	107.91 ms	k6 is 1.9× closer
Peak resident memory	841.4 MB	320.5 MB	k6 uses 2.6× less
Peak OS threads	1,037	15	k6 uses 69× fewer
CPU seconds consumed	28.19	21.08	k6 uses 1.34× less
Requests completed	69,027	71,343	Equivalent
Throughput	765.3/s	783.5/s	Equivalent
Errors	0%	0%	Both correct
Recorded from the load-generator process. Both runs completed with zero errors.			

JMeter memory k6 memory



JMeter allocates its thread pool inside the first 10 seconds and climbs to 835 MB. k6 rises gently to 320 MB and flattens.

JMeter k6 True latency - 50 ms



Everything above the dashed line is instrument overhead. The gap widens as you move to the tail — which is precisely where service level agreements are decided.

04 The mathematics

Little's Law — the equation behind every load test

This is the one formula the whole team should know. It ties together the three numbers we ever care about, and it is what turns "we have 400 users" into a test configuration.

$$N = \lambda \times (R + Z)$$

N concurrent users · **λ** throughput in requests/second · **R** response time · **Z** think time between actions

Check against our own run: $N = 1,000$ VUs, $R + Z = 1.05$ s

$\lambda = N \div (R + Z) = 1,000 \div 1.05 = \mathbf{952 \text{ requests/second predicted}}$

k6's dashboard plateaus at **≈950/s**. The theory and the measurement agree.

Use it to size our real tests. For a service with 400 concurrent users, a 0.3 s response time and 5 s of think time between clicks:

$\lambda = 400 \div (0.3 + 5) = \mathbf{75 \text{ requests/second}}$

That is the number to drive the test with — not "400 virtual users", which would generate roughly 13× the real traffic and fail a service that is perfectly healthy.

Cost per unit of work

Raw totals flatter whichever tool did less. Dividing by work done removes that.

$$C_{\text{req}} = \text{CPU seconds} \div \text{requests}$$

$$M_{\text{vu}} = (\text{peak} - \text{baseline}) \div \text{VUs}$$

k6 $C = 21.08 \div 71,343 = \mathbf{0.296 \text{ ms CPU per request}}$ | $M = (320.5 - 148.6) \div 1000 = \mathbf{0.172 \text{ MB per VU}}$

JMeter $C = 28.19 \div 69,027 = \mathbf{0.408 \text{ ms CPU per request}}$ | $M = (841.4 - 85.0) \div 1000 = \mathbf{0.756 \text{ MB per VU}}$

Ratio **1.38× CPU** and **4.40× memory** per unit of load

Subtracting the baseline is deliberate: it removes JMeter's pre-allocated 1 GB heap from the comparison and measures only what each additional user actually costs. The gap

grows, not shrinks, once you are fair to JMeter about its heap.

What that means at larger scale

$$\text{Memory}(n) \approx \text{baseline} + n \times M_{vu}$$

At **10,000 users** – $k6 \approx 1.8 \text{ GB}$ | JMeter $\approx 7.5 \text{ GB}$

Linear extrapolation only – not measured. JMeter would in practice hit thread-scheduling limits before memory.

05 Why the difference exists

None of this is a tuning accident. It follows from one design decision taken twenty years apart.

PEAK OS THREADS TO DRIVE 1,000 VIRTUAL USERS

JMeter – 1,037 threads

k6 – 15 threads

Bars to scale

JMeter creates one operating-system thread per virtual user. k6 multiplexes all 1,000 users across a handful of threads.

Waiters and diners

JMeter hires one waiter per diner. A thousand diners means a thousand waiters crowding one kitchen — most standing still, waiting on food, while the kitchen jams with bodies.

k6 employs fifteen waiters who never stand still: take an order, move to the next table, return when the food is up.

Same meals served. **1,037 staff versus 15.**

Why the crowded kitchen reads slow

When a waiter finally collects a plate, they must fight through the crowd before writing down the time. That queueing lands in the number they record.

This is *why* JMeter's reading drifts further from 50 ms, and why its worst case doubles.

Blueprint versus photocopy

A JMeter test is 800+ lines of machine-written XML. Nobody can review a change to it — you approve it because it ran, not because you read it.

A k6 test is 50 lines of JavaScript, reviewed in a pull request like any other code.

Tests we cannot review are tests we cannot trust.

The queue that measures itself

Time a restaurant queue by clocking only the people who got seated, and you miss everyone who gave up and left. Load tools do this when they wait for a slow reply before sending the next request.

Both tools have this. Both can fix it. It is a **configuration** choice, not a tool choice — and worth knowing before anyone raises it.

06 JMeter's flaws, ranked by how much they cost us

01 Coarser measurement — the one that matters most

Its error against a known 50 ms target was 1.7× k6's, and 1.9× at the worst case. A load test's entire purpose is the accuracy of that number.

measured · 61.66 ms vs 56.83 ms · max 206 ms vs 108 ms

02 One OS thread per user

A structural ceiling, not a setting. Industry guidance is 200–300 threads per JMeter engine; beyond that you must run a cluster of machines to do what one k6 process does alone.

measured · 1,037 threads vs 15 · 0.756 MB vs 0.172 MB per user

03 Tests that cannot be code-reviewed

Test plans are machine-generated XML. Diffs are unreadable and merge conflicts between two testers editing one plan are severe. This blocks the practice we most want to build.

structural · 800+ lines XML vs ~50 lines JavaScript

04 The GUI cannot be used for real tests

JMeter's own documentation instructs you not to run load tests from the GUI. So its headline advantage — the visual builder — is for authoring only; every real run is command line regardless.

Apache JMeter best-practices documentation

05 Silent result corruption

Leave a listener enabled during a load test and JMeter's overhead balloons, inflating the very numbers you are measuring. The symptom looks exactly like a slow application, which is how teams chase phantom bugs.

Apache JMeter best-practices documentation

06 Memory tuning is mandatory

Ships with a fixed 1 GB heap. Past a few hundred threads you hit OutOfMemory unless you know to edit the startup script — a trap that catches every newcomer.

verified · `-Xms1g -Xmx1g` default in `jmeter.bat`

07 No release in two and a half years

Latest stable is 5.6.3 from January 2024. The repository still receives commits, but shipped releases have stalled while k6 has moved through two major versions.

verified · [GitHub releases](#) [API](#)

08 Real logic means Groovy inside XML

The moment a test needs a condition, a calculation, or a token extracted from a response, you write a scripting language embedded in a GUI field — harder than the JavaScript would have been, and slower at runtime if written poorly.

`structural`

07 Where the industry is going

Tooling choices are also hiring and support choices. These figures were pulled live from the GitHub API on the day of writing.

K6 STARS

31,292

grafana/k6

JMETER STARS

9,517

apache/jmeter

STAR RATIO

3.3×

in k6's favour

LATEST RELEASE

k6 v2.x

JMeter 5.6.3 — January 2024

SIGNAL	APACHE/JMETER	GRAFANA/K6
Stars	9,517	31,292
Forks	2,286	1,610
Open issues	971	788
Last commit	14 Aug 2026	21 Aug 2026
Last stable release	Jan 2024	2026
Implementation	Java	Go
GitHub API, 22 August 2026. Fork count is included for completeness — it favours JMeter and largely reflects its greater age.		

Stated fairly: both repositories are alive — JMeter had a commit last week. The meaningful difference is not activity but *delivery*: JMeter has shipped no stable release since January 2024, while k6 has moved through two major versions. And 3.3× the stars means more answers when we are stuck, and more candidates who already know the tool.

08 Side by side

CRITERION	APACHE JMETER 5.6.3	GRAFANA K6 V2.1.0	WINNER
Measurement accuracy <i>(measured)</i>	23.3% error	13.7% error	k6
Worst-case accuracy <i>(measured)</i>	206 ms for a 50 ms call	108 ms	k6
Memory per user <i>(measured)</i>	0.756 MB	0.172 MB	k6
CPU per request <i>(measured)</i>	0.408 ms	0.296 ms	k6
Concurrency model	1 OS thread per user	Goroutines, multiplexed	k6
Users per machine	~200–300 recommended per engine	30,000–40,000 documented	k6
Test format	Generated XML	JavaScript	k6
Code review of tests	Impractical	Normal pull request	k6
CI pass/fail gates	Plugins and glue	Built-in thresholds, exit codes	k6
Scripting complex logic	Groovy inside XML	JavaScript	k6
Release cadence	None since Jan 2024	Active	k6
Community size	9,517 stars	31,292 stars	k6
Protocols beyond HTTP	JDBC, JMS, LDAP, FTP, SMTP	HTTP, gRPC, WebSocket, browser	JMeter
Record and replay	Built-in proxy recorder	None	JMeter
Free distributed testing	Built in, plain VMs	Needs Kubernetes or paid cloud	JMeter
Non-programmer friendly	Visual builder	Code only	JMeter
Version stability	Very stable	Breaking changes across majors	JMeter

Rows marked “measured” come from our own benchmark; the rest are verifiable properties of each tool.

JMeter wins five rows — and not one of them touches our work. Our APIs are HTTP and JSON, so the protocol advantage never opens. At 300–500 users a single machine is ample, so distributed mode never opens. Our team reads JavaScript daily. Only one k6 weakness can reach us — breaking changes between major versions — and pinning the version in our repository closes it.

09 What we are not claiming

A recommendation that only argues one side invites someone to find what was left out. These are the limits of this study, stated before anyone else states them for us.

- **JMeter is not inaccurate or unfit.** It completed 69,027 requests with zero errors and reported them honestly. It is the coarser and more expensive instrument, not a broken one.
- **At our scale, capacity is not the argument.** Both tools idled at roughly a third of one core. Anyone choosing on raw throughput at 500 users is optimising the wrong thing.
- **Part of JMeter's memory is its 1 GB pre-allocated heap,** not per-user cost. Our per-user figures subtract the baseline precisely to be fair about this.
- **JMeter wrote a 7.4 MB results file** during the run; k6 wrote only a summary. That is real disk work k6 did not do.
- **Generator and target shared one 8-core machine.** Neither was starved, but this is not an isolated laboratory.
- **1,000 users is not where JMeter struggles.** The divergence grows at 5,000–10,000+, which we did not test and do not claim.
- **One run each.** Repeat runs would tighten confidence, particularly in the CPU figure.

Independent corroboration. A separately published comparison recorded JMeter at 760 MB against k6 at 256 MB. Ours gave 841 MB against 320 MB — different hardware, the same ratio. The memory finding is not an artefact of our setup.

When we would still reach for JMeter. If we need to load-test a database directly over JDBC, a message queue over JMS, or drive load from a dozen plain virtual machines without Kubernetes. In that case we would add JMeter for that job rather than migrate everything — many teams run both.

10 Sources

Primary data is our own and reproducible from the harness published with this report. The following corroborate the architectural claims, guidance figures and version history.

[k6 — Running large tests \(official: memory per VU, 30,000–40,000 VUs per machine\)](#)

[Apache JMeter — Best Practices \(official: non-GUI mode, listeners, thread sizing\)](#)

[Apache JMeter releases \(latest stable 5.6.3, January 2024\)](#)

[Grafana k6 releases \(v2.x line\)](#)

[GitHub API — grafana/k6 \(star and activity figures\)](#)

[GitHub API — apache/jmeter \(star and activity figures\)](#)

[BlazeMeter — Maximum concurrent users in JMeter \(200–300 threads per engine\)](#)

[JVM thread stack size — a myth debunked](#)

[ScyllaDB — On coordinated omission](#)

[Red Hat Performance — Coordinated omission](#)

[Distributed load testing on Kubernetes: JMeter, k6 and Gatling](#)

[k6 vs JMeter benchmarked on a real app \(self-described as directional\)](#)

Measurements taken 22 August 2026. The harness — `target.js`, `test.k6.js`, `test.jmeter.jmx`, `measure.ps1` — together with the raw results is published alongside this report so any figure here can be reproduced and checked.